

# contexture

*User manual*

## Contents

|                                                                                    |    |
|------------------------------------------------------------------------------------|----|
| 1. What contexture is for .....                                                    | 2  |
| 2. Installation and compiling .....                                                | 2  |
| 3. Quickstart: a manuscript with a generated companion .....                       | 2  |
| 4. The anchor primitive: <code>anchor</code> , <code>anchors</code> .....          | 4  |
| 4.1. Re-emitting stored content: <code>strip-labels</code> .....                   | 5  |
| 4.2. Structural utilities .....                                                    | 6  |
| 5. A fuller example: list of figures .....                                         | 6  |
| 6. <code>satellite</code> and <code>bundle</code> : the shared pilot .....         | 8  |
| 7. Two independent compile axes: <code>variant</code> , <code>preview</code> ..... | 9  |
| 8. Diagnostics: <code>diagnose</code> , <code>set-strict</code> .....              | 11 |
| 9. <code>xref</code> : cross-references with a real page number .....              | 12 |
| 10. Composing independent packages .....                                           | 12 |
| 11. Where to go next .....                                                         | 13 |

## 1. What contexture is for

Typst can compile one source file into several output documents at once (the experimental bundle export, `--features bundle --format bundle`) that all share one introspection space: a `query()` run from any one of them sees content laid out in **all** of them, with real, final page numbers, because they were genuinely composed together in the same pass.

contexture is a small toolkit built directly on that capability:

- `anchor` / `anchors` — mark a spot in one document, read it back from any other, by real page.
- `satellite` / `bundle` — one shared entry point that decides which documents come out of a compile, so several independent pieces of code can each contribute a document without fighting over how the split works.
- `variant` / `preview` — two small flags any document you build can read, to change what a compile produces without adding a new one-off command-line flag for each idea.
- `diagnose` / `set-strict` and `xref` — a couple of shared conveniences: a visible way to flag a problem, and a `@label` that also prints its real page number.

contexture itself has no opinion on what any of this is **for** — no notion of revisions, checklists, glossaries, or any other specific kind of document. Every example in this manual is self-contained: a manuscript plus a couple of small functions written directly against these primitives, nothing else involved, to show that the toolkit stands on its own. Only the closing chapter, “Composing independent packages,” brings in outside packages — not because this manual needs them to make its point, but to show what it looks like when two packages built independently on contexture end up sharing a compile.

## 2. Installation and compiling

```
#import "@preview/contexture:0.1.0": *
```

Producing more than one document from a single file needs Typst’s bundle export:

```
typst compile --features bundle --format bundle main.typ
```

Every project built on contexture can also be compiled with two extra flags, explained in full in “Two independent compile axes” below:

```
typst compile --features bundle --format bundle --input variant=... main.typ
typst compile --features bundle --format bundle --input preview=true main.typ
```

## 3. Quickstart: a manuscript with a generated companion

The simplest thing to build on contexture: a manuscript, plus a second document generated from it. Below, a two-function glossary — `term()` marks where a term is first defined; `render-glossary()` lists every term found anywhere in the bundle, each with the real page it landed on. Neither function is part of contexture — this is what a package built on it looks like, complete:

```
#import "../..lib.typ": *

// A tiny satellite package, built from scratch out of contexture's
// primitives alone. `term()` anchors a short definition where it's
// first used; `render-glossary()` lists every one of them, in document
// order, with its real page number.

#let term(id, body) = {
  anchor("demo-term", (id: id, body: body))
  body
}

#let render-glossary() = context {
  let hits = anchors("demo-term")
  if hits.len() == 0 {
    [No terms defined.]
  }
}
```

```

    } else {
      for h in hits [
        *#h.value.id* --- #h.value.body (p. #h.location().page())
        #linebreak()
      ]
    }
  }
}

#let glossary = satellite("glossary", render: () => render-glossary())

#show: bundle.with(
  template: body => {
    set page(width: 16.6cm, height: auto, margin: 12pt)
    set text(size: 10.5pt)
    body
  },
  documents: (glossary,),
)

= Introduction

The trial used #term("itt")[intention-to-treat] analysis throughout.

= Methods

Randomisation used #term("block")[permuted blocks] of size 4.

```

Compiling this exact file — `bundle-glossary-basics.typ`, the one shown above, nothing else — with `typst compile --features bundle --format bundle` produces two PDFs. The first is `manuscript.pdf`:

## Introduction

The trial used intention-to-treat analysis throughout.

## Methods

Randomisation used permuted blocks of size 4.

That name has nothing to do with the file just compiled, which could be called anything (`main.typ`, `report.typ`, ...) — it comes entirely from `bundle()`'s own `manuscript-name:` parameter, "manuscript" by default. More on this in “satellite and bundle” below.

`glossary.pdf`, from the very same compile:

`itt` — intention-to-treat (p. 1)  
`block` — permuted blocks (p. 1)

Three things are happening, and the rest of this manual is one chapter per thing:

1. `term()` calls `anchor("demo-term", ...)` — drops a small, named piece of data at this exact spot, then renders body as usual. Covered next, in “The anchor primitive.”
2. `render-glossary()` calls `anchors("demo-term")` — every anchor of that kind, anywhere in the bundle, each with a real `location()` to read a page number off. Same chapter.
3. `satellite("glossary", ...)` describes the second document; `#show: bundle.with(documents: (glossary,))` is what actually produces both PDFs from one compile. Covered in “satellite and bundle.”

If what brought you here is specifically the `variant/preview` flags, skip ahead — they get their own chapter, with a dedicated example, further down.

This same shape — an anchor at each interesting spot, a satellite that queries them — covers more than a glossary: a list of figures or tables (the next worked example), an index of defined terms, an answer key kept apart from the exam it belongs to, an executive summary that cites the real page of whatever it’s summarizing, supplementary material that cites “as shown in Figure 3, p. 7” of a document compiled in the very same pass. Nothing app-specific has to live inside `contexture` for any of these to work — each is the same handful of primitives, arranged differently.

## 4. The anchor primitive: `anchor`, `anchors`

`anchor(kind, payload)` marks the current location with a namespaced, queryable piece of data. `kind` namespaces the anchor so two unrelated pieces of code picking the same `id` scheme never collide — prefix it with something specific to what you’re building (“`glossary-term`”, “`figure-list-entry`”). `payload` is whatever you need back later — entirely opaque to `contexture` itself. Deliberately renders nothing on its own beyond the metadata: emitting it and deciding how (or whether) to render content around it are two different jobs, left to the caller, exactly as `term()` above does both explicitly.

`anchors(kind)` every anchor of that kind, in document order, from **anywhere in the bundle** — including a document other than the one this is called from, which is the entire point. Must be called from within a context.

#### 4.1. Re-emitting stored content: `strip-labels`

A term's body can be arbitrary content, including a labelled figure — and re-emitting that figure verbatim into the glossary would otherwise plant a second copy of its label, which Typst rejects outright as a duplicate. `strip-labels(node)` reconstructs `node` with every label removed, and — specifically for a figure, a labelled block equation, or a heading — pins the **real**, already-resolved number of the true original onto the copy's own numbering, read directly via a fresh `query()` of that label. The copy shows the same number as the original; only the original stays a real, referenceable target.

```
#import "../lib.typ": *

// Same idea as bundle-glossary-basics.typ, but one term's body is itself
// a labelled figure --- re-emitting it verbatim in the glossary would
// otherwise plant a second copy of <tab-doses>, which Typst rejects as a
// duplicate label. strip-labels() drops the label from the copy while
// pinning the real, original "Figure 1" number onto it, read directly
// off the true instance via a query --- so both copies show the same
// number, and only one of them is a real, referenceable target.

#let term(id, body) = {
  anchor("demo-term", (id: id, body: body))
  body
}

#let render-glossary() = context {
  for h in anchors("demo-term") [
    *#h.value.id* (p. #h.location().page()):
    #strip-labels(h.value.body)
    #v(0.5em)
  ]
}

#let glossary = satellite("glossary", render: () => render-glossary())

#show: bundle.with(
  template: body => {
    set page(width: 16.6cm, height: auto, margin: 12pt)
    set text(size: 10.5pt)
    body
  },
  documents: (glossary,),
)

= Methods

#term("doses")[
  #figure(
    table(columns: 2, [*Arm*], [*Dose*], [A], [10mg], [B], [Placebo]),
    caption: [Study drug doses.],
  ) <tab-doses>
]

As shown in @tab-doses, two arms were compared.
```

Both the manuscript and the glossary show “Table 1” — the genuine, resolved number, not a guess:

## Methods

| Arm | Dose    |
|-----|---------|
| A   | 10mg    |
| B   | Placebo |

Table 1: Study drug doses.

As shown in Table 1, two arms were compared.

*manuscript.pdf*

doses (p. 1):

| Arm | Dose    |
|-----|---------|
| A   | 10mg    |
| B   | Placebo |

Table 1: Study drug doses.

*glossary.pdf*

## 4.2. Structural utilities

`collect-metadata(body, tag)` and `collect-labels(body)` are the structural building blocks `strip-labels` and `anchor/anchors` themselves are built from — a plain walk of an in-memory content tree, no `context` or `layout` involved, exposed directly for code that needs to inspect content it's holding but hasn't (or may never) placed into any document this compile. `is-blank(body)` answers “does this contain any real text at all” (handy for flagging an accidentally empty call to something like `term()`); `is-textual(body)` answers “is this safe to wrap in literal quotation marks”, i.e. free of any `figure`, `table`, or block equation that a stray quote mark would otherwise float above.

## 5. A fuller example: list of figures

The glossary above makes the mechanism easy to follow, at the cost of being a little toy — two terms, one line each. Here's a version closer to what you'd actually paste into a real project: a “List of Figures” companion, several entries deep, spanning a page break, each citing the real page its figure landed on.

```
#import "../lib.typ": *

// A fuller example than the glossary above -- closer to something you
// might actually copy into a real project: a "List of Figures"
// companion, generated instead of hand-maintained, citing the real page
// each figure landed on. Numbers figures in anchor order (1, 2, 3, ...)
// rather than reading Typst's own figure counter -- simpler, and exact
// here since fig() is the only thing creating figures in this document;
// a project mixing fig() with bare figure() calls would want
// counter(figure).at(..) instead.

#let fig(caption, body) = {
  let content = figure(body, caption: caption)
  anchor("demo-figure", (caption: caption))
  content
}

#let render-list-of-figures() = context {
```

```

let hits = anchors("demo-figure")
for (i, h) in hits.enumerate() [
  *Figure #(i + 1).* #h.value.caption --- p. #h.location().page()
  #linebreak()
]
}

#let list-of-figures = satellite("list-of-figures", render: () => render-list-of-figures())

#show: bundle.with(
  template: body => {
    set page(width: 16.6cm, height: auto, margin: 12pt)
    set text(size: 10.5pt)
    body
  },
  documents: (list-of-figures,),
)

= Methods

#fig([Study flow diagram.], rect(width: 3cm, height: 1.5cm, align(center + horizon)[Flow]))

#lorem(90)
#pagebreak()

= Results

#fig([Primary outcome over time.], rect(width: 3cm, height: 1.5cm, align(center + horizon)[Chart A]))

#fig([Subgroup analysis.], rect(width: 3cm, height: 1.5cm, align(center + horizon)[Chart B]))

```

manuscript.pdf — three ordinary figures across two pages, nothing about them different from any other Typst document:

## Methods

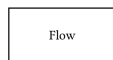


Figure 1: Study flow diagram.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

## Results

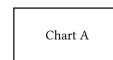


Figure 2: Primary outcome over time.

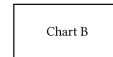


Figure 3: Subgroup analysis.

page 2

page 1

list-of-figures.pdf, from the same compile:

**Figure 1.** Study flow diagram. — p. 1

**Figure 2.** Primary outcome over time. — p. 2

**Figure 3.** Subgroup analysis. — p. 2

`fig()` numbers its own entries by the order `anchor()` calls arrive in, rather than reading Typst's built-in `counter(figure)` — simpler, and exactly right as long as `fig()` is the only thing creating figures in the document (a project mixing `fig()` with bare `figure()` calls would want `counter(figure).at(hit.location())` instead, the same real-counter trick `strip-labels` uses above). Either way, the page numbers are never guessed or hand-typed — they come from `location().page()` on the anchor Typst itself placed, in the very compile that produced `manuscript.pdf`.

## 6. `satellite` and `bundle`: the shared pilot

`document(...)` — Typst's own primitive for naming one output of a bundle compile — cannot be nested inside another `document(...)` call. That rules out letting several independent pieces of code each call `document(...)` on their own: whichever runs second would be trying to nest its document inside whatever the first one already produced. `contexture.bundle` is the fix: the **only** place that ever calls `document(...)`. Anything built on `contexture` instead exposes a small constructor that returns a `satellite(...)` value — inert data, not a `document(...)` call — and the author lists as many of those as they like under one shared `documents:`, exactly as the quickstart above already did with `glossary`.

`satellite(name, render:, applicable:, side-content: none)` describes one document to build alongside the manuscript. `name` is the base filename (`bundle` appends the variant/preview suffix, see next chapter).



`render()` -> content produces this document's content, called only when `applicable()` -> `bool` (default: always) says yes for the current compile.

`bundle(template:, documents: (), strict: false, manuscript-name: "manuscript", body)` the pilot itself, called via `#show: bundle.with(...)` — `body` is the rest of the document, typically `#include "manuscript.typ"`. Builds the manuscript (`template(body)`) plus every satellite whose `applicable` returns true, each as its own real document sharing this one compile's introspection space with all the others.

`manuscript-name:` is the **only** thing that decides the manuscript's output filename — it has no connection at all to the name of whatever `.typ` file you actually run `typst compile` on (every example in this manual compiles the snippet file shown directly, e.g. `bundle-glossary-basics.typ`, and still produces `manuscript.pdf`). A project that keeps its manuscript's prose in its own file, `#included` into `body`, is free to name that file anything — calling it `manuscript.typ` is only a common convention, not a requirement `bundle()` checks for. Every example in this manual takes the simplest route instead: the file you see printed **is** the file compiled, with the manuscript's own content written directly after `#show: bundle.with(...)`, no separate `#include` at all.

**Restricting a compile to fewer documents.** `--input only=<comma-separated satellite names>` restricts a single compile to the manuscript plus just the named satellites — `--input only=` with nothing after it produces the manuscript alone, whatever `documents:` lists. A command-line choice for a fast preview, not a property of the project: naming a satellite under `only:` that declines to build itself this compile (its own `applicable` says no) doesn't force it to.

`side-content.` Content a satellite wants placed in the manuscript regardless of whether **it itself** gets built this compile. Useful when a satellite defines anchors that something elsewhere in the manuscript depends on for a check of its own (e.g. “does this passage have a matching answer somewhere?”) — skip the satellite via `only=` and, without `side-content`, that check would wrongly report every single one of them as missing, purely because the satellite that would have answered them wasn't built this time. Paired with `collect-anchors(body, kind)` — the structural counterpart to `anchors`, finding anchors already sitting inside an in-memory `body` that was never placed into any document — and `reemit(collected)`, which registers one such found anchor as if it had been placed here.

## 7. Two independent compile axes: variant, preview

Two flags are available to any document built on `contexture`, read by two plain functions:

`variant()` which **version of the truth** this compile is producing — an intrinsic property of what the content itself means. “plain” by default (`--input variant=...` to change it); a project defines whatever other values make sense to it and reads them back with `variant() == "..."`. Code with no notion of variants never touches this at all, and always sees “plain”.

`preview()` `true/false`, from `--input preview=true`. A drafting aid only: “show me my own working, temporarily” — never a reason to change what a document's content **means** (that's `variant`'s job), only how much of the plumbing is made visible while writing.

The difference in one sentence: `variant` decides **whether something is there at all**; `preview` decides **how much you can see of how it got there**. They're independent on purpose, so a project can cross them freely rather than picking one axis and losing the other.

A small worked example, built on the same `term()` as the quickstart, plus one new function that only exists to make the difference concrete:

```
#import "../lib.typ": *

// Same term() as the glossary quickstart, now doing one more thing:
// highlighting its own body whenever preview() is on -- a debug view of
// where every anchor sits, never part of the real deliverable.
//
// note() is new: an aside that only renders at all -- box, text, and
// all -- when variant() is "internal". Its anchor is still registered
// every time, so open-notes below can always find it, but its content
// only *exists* in the internal variant. That's the difference between
// the two axes: preview changes how something already there is shown;
```

```
// variant decides whether it's there at all.

#let term(id, body) = {
  anchor("demo-term", (id: id, body: body))
  if preview() {
    box(fill: yellow.lighten(80%), inset: 2pt, radius: 1pt)[#body]
  } else {
    body
  }
}

#let note(body) = {
  anchor("demo-note", (body: body))
  if variant() == "internal" {
    box(fill: red.lighten(90%), inset: 4pt, radius: 2pt, width: 100%)[
      #text(style: "italic", size: 0.9em)[Reviewer note: #body]
    ]
  }
}

#let render-open-notes() = context {
  let hits = anchors("demo-note")
  if hits.len() == 0 {
    [No open notes.]
  } else {
    for h in hits [- #h.value.body (p. #h.location().page())]
  }
}

#let open-notes = satellite(
  "open-notes",
  render: () => render-open-notes(),
  applicable: () => variant() == "internal",
)

#show: bundle.with(
  template: body => {
    set page(width: 16.6cm, height: auto, margin: 12pt)
    set text(size: 10.5pt)
    body
  },
  documents: (open-notes,),
)
```

#### = Methods

The trial used `#term("itt")`[intention-to-treat] analysis throughout.

`#note`[Double-check this matches the pre-registration.]

Randomisation used `#term("block")`[permuted blocks] of size 4.

`term()` highlights its own body whenever `preview()` is on — a debug view of where the anchors are, nothing else changes. `note()` is a document aside that only renders — box, text, and all — when `variant()` is "internal"; its anchor is still registered on every compile, so `open-notes` can always find it, but its content genuinely doesn't exist outside the internal variant. `open-notes` itself only builds under `variant() == "internal"` — a document listing every open note has nothing to say about a variant that has none.

Compiling this one file four ways produces four different, genuinely independent combinations:

| Compile                                       | Files produced                                                   |
|-----------------------------------------------|------------------------------------------------------------------|
| (no flags)                                    | manuscript.pdf                                                   |
| --input variant=internal                      | manuscript-internal.pdf, open-notes-internal.pdf                 |
| --input preview=true                          | manuscript-preview.pdf                                           |
| --input variant=internal --input preview=true | manuscript-internal-preview.pdf, open-notes-internal-preview.pdf |

### Methods

The trial used intention-to-treat analysis throughout.  
Randomisation used permuted blocks of size 4.

(no flags)

### Methods

The trial used intention-to-treat analysis throughout.

Reviewer note: Double-check this matches the pre-registration.

Randomisation used permuted blocks of size 4.

variant=internal

### Methods

The trial used **intention-to-treat** analysis throughout.  
Randomisation used **permuted blocks** of size 4.

preview=true

### Methods

The trial used **intention-to-treat** analysis throughout.

Reviewer note: Double-check this matches the pre-registration.

Randomisation used **permuted blocks** of size 4.

both together

Notice what stays constant across each pair: turning `preview` on never makes the reviewer note appear — that’s `variant`’s call, not `preview`’s — and switching to the internal variant never highlights the terms on its own. Only the fourth compile, with both flags, shows both effects at once, each exactly as it looks alone. That independence is the entire reason these are two flags rather than one shared string: a single flag can only ever represent one axis at a time, and code that treats “anything other than the default” as “my own alternate behavior is on” has no way to tell **which** alternate behavior a caller meant. Two flags, two distinct questions, make that ambiguity impossible rather than merely avoided by convention.

`bundle` folds both axes into every filename independently, in the same orthogonal way — so a `preview` compile never silently overwrites the plain deliverable on disk, whatever `variant` happens to be at the same time, and a compile using both is simply both suffixes, in order.

## 8. Diagnostics: `diagnose`, `set-strict`

Typst has no public API to emit a soft compiler warning from user code, so the closest available approximation is a visible marker rendered directly at the fault location, which most Typst editors preview live.

`diagnose(message, always: false)` reports a problem at the call site. Under `set-strict(true)`, always a hard panic — a real compile error, in any mode. Otherwise, a visible inline marker — muted specifically when `variant() == "plain"` and `preview()` is off (`always: false`, the default), since that’s the file most likely to leave this codebase and reach someone who never asked to see an internal warning; shown unconditionally when `always: true`, for a diagnostic embedded in a document that is **never** itself the deliverable (a generated report, an internal checklist) where muting it would mean it’s never seen at all.

`set-strict(v)` turns every `diagnose(...)` call, anywhere in the bundle, into a hard error at once — one shared CI gate rather than a check per document. Normally set via `bundle(strict: true, ...)`, not called directly.

`xref` (below) always passes `always: true` — a broken cross-reference should never be silently invisible even in the real, submitted deliverable:

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
#set heading(numbering: "1.")

= Results <sec-results>

#figure(table(columns: 2, [*Arm*], [*n*], [A], [120], [B], [118]), caption: [Participants per arm.]) <tab-n>

See #xref(<tab-n>) and #xref(<sec-results>) above. This one doesn't exist: #xref(<tab-missing>).
```

## 1. Results

| Arm | n   |
|-----|-----|
| A   | 120 |
| B   | 118 |

Table 1: Participants per arm.

See Table 1, p. 1 and Section 1, p. 1 above. This one doesn't exist: ! xref(<tab-missing>): not found.

## 9. xref: cross-references with a real page number

`xref(label)` behaves like `@label/ref(label)` — which already resolves across documents in a bundle, a satellite's own `@tab-results` renders the manuscript's real "Table 3" — but appends the real page number: "Table 3, p. 14". Explicit rather than a bare `@label`, so it stays correct even if a future document duplicates the same label, where a bare `ref` would become ambiguous between the two copies. It operates on plain Typst labels (figures, headings, equations, ...), not on `contexture.anchor()` — a separate, narrower tool for the common case where a real Typst label already exists and only the page number needs adding.

## 10. Composing independent packages

Everything above is self-contained — no example so far needed anything beyond `contexture` itself. In practice, `contexture` is meant as a shared foundation that several packages build on at once. `@preview/palimpsest` (manuscript revision letters) and `@preview/checkitoff` (reporting-guideline checklists) are two such packages, published independently of each other and of `contexture`, neither aware the other exists. Combining them needs nothing beyond listing both of their satellites under the same documents: — the scenario the whole design exists for:

```
#import "../.../typst-palimpsest/lib.typ" as palimpsest
#import "../.../typst-checkitoff/lib.typ" as checkitoff
#import "../.../lib.typ": bundle

// The flagship "composition" example: two independently-developed
// packages, each built on contexture and neither aware the other
// exists, combined in one compile through nothing more than one shared
// `documents` list. `letter(...)` and `checklist(...)` are each just a
// `satellite(...)` value; `bundle(...)` (this package, not either of
// theirs) is the only thing that ever calls `document(...)`.

#let tiny-checklist = (
  name: "TINY",
  full-name: [Tiny reporting checklist],
  items: (
    (section: "Methods", topic: "Randomisation", group: none, id: "1",
      description: [How the allocation sequence was generated.]),
    (section: "Methods", topic: "Outcome assessment", group: none, id: "2",
      description: [How the primary outcome was assessed.]),
  ),
)

#let my-template(body) = {
  set page(width: 16.6cm, height: auto, margin: 12pt)
  set text(size: 10.5pt)
  body
}

#let exchanges = palimpsest.reviewer(1)[
  #palimpsest.exchange(<r1-1>)[Please clarify whether outcome assessment was blinded.][
    Blinding is now specified. #palimpsest.pinpoint(<r1-1>)
  ]
]

#show: bundle.with(
  template: my-template,
  documents: (
```

```

    palimpsest.letter(exchanges: exchanges),
    checkitoff.checklist(checklist: tiny-checklist),
  ),
)

// Item 1 has nothing to do with this reviewer exchange -- check() just
// renders its own text at its own spot, independently of passage().
#checkitoff.check("1")[Randomisation used a computer-generated sequence.]

// Item 2, though, IS the reviewer exchange: the added text is both a
// tracked revision AND the manuscript's answer to item 2. check(id, body)
// and passage() would each render their own body -- used together on
// the exact same span, that would print it twice. check(id)'s bare,
// point-marker form registers item 2's coverage without rendering
// anything; passage() is the only call that actually prints the text,
// once, and handles its tracked-mode marks.
#palimpsest.passage(<r1-1>)[
  The primary outcome was assessed #palimpsest.add[by a rater blinded to group assignment].
]
#checkitoff.check("2")

```

One compile, four ways, exactly like “Two independent compile axes” above, now with two real packages instead of a self-contained demonstration:

| Compile                                      | Files produced                                               |
|----------------------------------------------|--------------------------------------------------------------|
| (no flags)                                   | manuscript.pdf, response.pdf, checklist.pdf                  |
| --input variant=tracked                      | manuscript-tracked.pdf, response-tracked.pdf                 |
| --input preview=true                         | manuscript-preview.pdf, response-preview.pdf                 |
| --input variant=tracked --input preview=true | manuscript-tracked-preview.pdf, response-tracked-preview.pdf |

The last compile shows both packages’ overlays together, in the same document, each independent of the other:

Randomisation used a computer-generated sequence. <sup>[1]</sup>

The primary outcome was assessed by a rater blinded to group assignment. <sup>[R1-1]</sup> <sup>[2]</sup>

Building a marking function that renders its own content, the way `passage()` and `check()` both do here, raises exactly one extra question that a single-package example never has to answer: what happens when two such functions from two different packages meet on the same document, or even the same span? Two rules keep that safe — never nest one package’s marking function inside another’s, and never call two of them as independent, rendering siblings on the exact same span (one of them needs a body-free form instead, to register coverage without printing the text twice). Both rules, why they’re necessary, and the exact two-shape pattern (`check(id)` vs. `check(id, body)`) that resolves the second one, are documented where they belong: in `palimpsest`’s and `checkitoff`’s own manuals, each under a section called “Combining with another `contexture` package.”

## 11. Where to go next

This manual covers `contexture` on its own — everything above works with no other package installed. For what to build with it:

- Manuscript revisions and a reviewer response letter that cites the real pages: `@preview/palimpsest`’s own manual.
- Reporting-guideline checklists (CONSORT, PRISMA, SPIRIT, STARD, STROBE) that cite the real pages: `@preview/checkitoff`’s own manual.
- Combining several such packages in one compile: “Composing independent packages” above, and the “Combining with another `contexture` package” section in each package’s own manual.